TAKKE

Website Design and Development

Project Report, Design Document and Test Plan

Website: http://takke.infinityfreeapp.com/

Contents

Part 1: Project Report **3**

1. DNS 3

2. Communication Protocols, Server Hardware, Operating System and Web Server Software 4

3. Website Development Technologies, Tools, Techniques and Frameworks 6

4. Search Engine Optimisation 9

5. Front-End and Back-End Website Technologies 11

6. Online Website Creation Tools vs Custom-Built Websites 12

7. Design and Development Process 14

Hosting 16

References 17

Part 2: Website Design Document **18**

Website Goal 18

Scope 18

Requirements 19

Sitemap 19

Wireframes 20

Part 3: Test Plan **23**

Scope 23

Schedule 24

Test Cases 24

Evaluation of the Quality Assurance (QA) Process 28

Part 1: Project Report

1. DNS

1.1 Types and Purpose of DNS

DNS (Domain Name System) is a networking system that maps domain names to IP addresses. A domain name is a unique identifier for an IP address: when a domain name is typed into the URL bar, DNS converts it into the IP address that the computer understands. DNS consists of a large number of servers, organised in a hierarchy and distributed around the world. The hierarchy has root-level, country-level, top-level and secondary-level domains, plus the service. For example, in www.htu.edu.jo, www is the service, htu is the secondary-level domain, edu is the top-level domain and jo is the country-level domain.

DNS has several types of servers:

- **Recursive resolver:** a server that acts as the middleman between the user and the other DNS servers. It receives the user's query and looks up the IP address connected to the domain name. If the IP address is stored, the server gives it to the user. If not, the server sends a request to other DNS name servers to find it.
- **Root name server:** acts like a library that points to the next step. The recursive resolver sends a request to it when it does not have the IP address. The root name server does not know the IP address, but it sends the query to the right top-level domain name server.
- **TLD (Top-Level Domain) name server:** manages requests based on the domain extension, such as .com, .org or .edu. When the query arrives and the extension is identified, the query goes to the authoritative name server to look up the specific domain.
- **Authoritative name server:** the final step in the DNS lookup process. It provides the IP address of the requested domain name and stores the DNS record for that domain.

An important part of DNS is the DNS lookup, which translates a domain name into an IP address.

The main purpose of DNS is the mapping between IP addresses and domain names. Because it is hard for humans to remember the IP addresses of websites, it is easier to remember domain names, so DNS makes it easier to navigate the internet. DNS also makes the internet more user-friendly, reliable and accessible, since it allows users to reach websites from anywhere in the world.

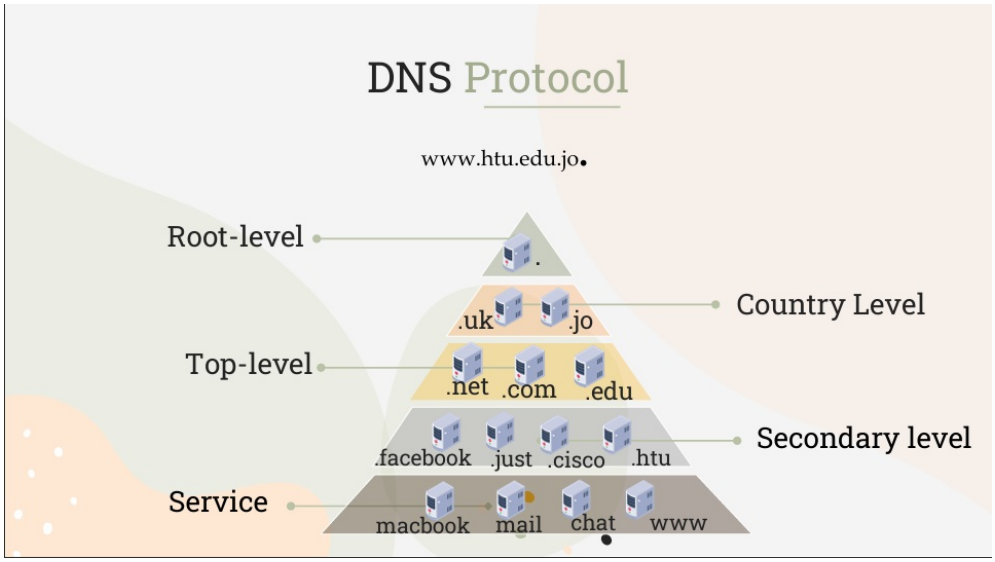
1.2 How Domain Names Are Organised and Managed

Domain names are managed in a hierarchical structure made of the root level, country level, top level, secondary level, and the service or hostname.

- **Root level:** shown as an empty string or a dot. It is the highest level and the starting point of the DNS hierarchy, and it is managed by the 13 root name servers worldwide.
- **Country level:** represented by the country code, such as jo. It indicates the location of the

website and is managed by organisations designated by each country.

- **Top level:** categorises domains by purpose, such as .com or .org. It is the last part of the domain name and is managed by organisations such as ICANN.
- **Secondary level:** the name of the website, such as google. It is managed by the owner of the domain.
- **Service or hostname:** the first part of the domain name, such as www, ftp or blog. It shows the type of service and is configured by the domain owner.



DNS hierarchy for www.htu.edu.jo

2. Communication Protocols, Server Hardware, Operating System and Web Server Software

2.1 Communication Protocols Definition

Communication protocols are a common set of rules that allow devices to communicate and transmit data over one or more networks. Each protocol has a specific purpose:

- **TCP/IP (Transmission Control Protocol/Internet Protocol):** used to transmit data over a network. It prepares the packets and forwards them over the network.
- **IP:** addresses packets so they can be routed to their destination.
- **FTP (File Transfer Protocol):** used to transfer files between a client and a server, for example uploading website files to a web server.
- **SMTP (Simple Mail Transfer Protocol):** responsible for sending emails from a client to a server.
- **POP (Post Office Protocol):** responsible for retrieving emails from a mail server.
- **UDP (User Datagram Protocol):** works at the transport layer and provides fast, connectionless data transmission. It is used for calls, video streaming and online games.
- **DHCP (Dynamic Host Configuration Protocol):** assigns IP addresses to devices automatically and dynamically, based on pools.
- **HTTP (Hypertext Transfer Protocol):** responsible for accessing web applications.

2.2 Relationship Between Protocols, Hardware, Operating System and Web Server Software

2.2.1 Purpose and Relationship

- **Protocols** allow devices to communicate and transmit data over networks.
- **Hardware** refers to the physical components of a server, such as the CPU, RAM and NIC. They provide the storage and power needed to handle requests.
- **Operating systems** such as Windows and Linux sit between the hardware and the software, and they manage the hardware so that it can run the software.
- **Web server software** such as Apache handles requests like HTTP/HTTPS GET, POST or PUT, and responds with a failure such as 404 or a success such as 200.

The operating system manages the hardware so it can run the software. Without it, the hardware could not run the web server software. Web servers use protocols such as HTTP/HTTPS to communicate with users, and the web server runs on the operating system, which provides the environment for the software to run. The hardware is the physical base that supports the operating system, the web server software and the protocols.

2.2.2 Relation to Designing, Accessing and Publishing the Website

Designing

- Protocols: they matter when testing whether the website uses HTTP or HTTPS. I also used FTP to transfer the website files from my computer to the hosting service.
- Hardware: I designed and tested the website on my own laptop. The hardware must be good enough to support the development tools.
- Operating system: it provides the environment for the development tools. I used Windows and developed the website in Visual Studio Code.
- Web server software: it simulates how the website will work on a live server. I used XAMPP for testing during development.

Accessing

- Protocols: when users request the website, their browser uses HTTP/HTTPS, and DNS translates the domain name into an IP address.
- Hardware: it must be good enough to display the website content and to handle the requests.
- Operating system: it must be reliable so the website stays accessible, and it provides the environment for the browser that opens the website.
- Web server software: it manages requests such as GET, POST or PUT from users and delivers the web page to the user's browser.

Publishing

- Protocols: FTP is used to upload the website files to the hosting service, and DNS makes sure that the domain name points to the correct IP address.
- Hardware: it is essential for uploading the files and hosting the website on a physical or

virtual server that can handle the traffic.
- Operating system: it makes sure the web server software can manage the resources.
- Web server software: it shows the requested page in the user's browser.

3. Website Development Technologies, Tools, Techniques and Frameworks

3.1 Website Development Technologies

Technologies are inventions, findings and innovations used to solve problems, achieve goals and create products.

3.1.1 Available Website Technologies

- **Browsers:** browsers such as Edge, Chrome or Brave allow users to access the website. They render websites differently, so older versions of some browsers may not support some tags or styles.
- **HTML and CSS:** HTML manages the content, while CSS manages the styling. CSS animations improve user engagement, and placing HTML content correctly makes the website easy to use.
- **Programming languages:** languages such as JavaScript allow dynamic validation on the sign-in and sign-up pages.
- **Frameworks and libraries:** frameworks such as Bootstrap provide reusable components and reduce development time.
- **Databases:** databases keep data organised and consistent across multiple users, and store all the data the website needs.
- **Protocols:** protocols such as HTTPS keep communication secure and protect sensitive data.
- **Data formats:** data formats make it easier to identify what a file is and improve file management.
- **APIs:** they make it easier to add new features or services to the website.

3.1.2 Technologies Used to Design, Develop and Manage My Website

In my website I used HTML and CSS, JavaScript, PHP, a database, Bootstrap and a browser for testing.

I used HTML and CSS to display the content and style it, including colours and fonts. I used different HTML form elements so users can enter data, and I wrote the code following best practices to make it easier to maintain and update. I used CSS hover effects on buttons and links, which improves user engagement.

I used JavaScript to show alerts when the user enters an invalid username or password, or when the user presses checkout. PHP was used for the back end: it connects to the database and inserts, reads, updates and deletes data, which makes the website dynamic. The database

stores and manages Takke's data.

Another technology I used is Bootstrap. I used the Bootstrap cards, which saved time and made the code easier to maintain. The browser I used to test the design, functionality and management of the website was Firefox Developer Edition.

3.2 Website Development Tools

Tools are the software used to develop and design websites, and they make it easier for developers to maintain their development environment.

3.2.1 Available Tools for Website Design and Development

- **Code editors:** tools such as Visual Studio Code and Atom are used to write and edit code. They have extensions such as Prettier and Live Server, which let developers see the website live while writing code and keep the code clean. They also have debugging tools that help developers identify errors.
- **Browser developer tools:** Firefox Developer Edition has tools for testing screen sizes, analysing the network and showing console errors.
- **Image optimisation tools:** tools such as Adobe Illustrator reduce file size and enhance quality, which speeds up loading.
- **Wireframe and sitemap tools:** tools such as Miro show the whole design and functionality of the website, saving time and resources.
- **Local environment:** tools such as Apache, phpMyAdmin and XAMPP allow the developer to test the website locally before publishing it.
- **WordPress:** a tool that allows people to create a website without coding knowledge.
- **Google Search Console:** finds issues such as broken links, helps improve search visibility and provides performance reports.

3.2.2 Tools Used to Design and Develop My Website

I used Visual Studio Code with the Live Server and Prettier extensions and its debugging tools. I also used the developer tools in Firefox Developer Edition, including the features described above, to make sure everything worked correctly.

I used Miro to design the wireframes and the sitemap. They gave me the foundation of the website, showed me its hierarchy, and turned the design in my mind into a visual one, so I was sure about how to develop it.

The most important tool was the local development environment, which includes XAMPP, Apache and phpMyAdmin. It let me test all the functionality of the website, including reading from the database and inserting, updating and deleting data, and it helped me manage all the database operations. My computer acted as both the client and the server.

3.3 Website Development Techniques

Techniques are ways of doing a specific task or writing code, such as responsive design.

3.3.1 Available Techniques for Website Design and Development

- **Responsive design:** makes all website components work well on all devices, so the developer does not need to make multiple versions of the website.
- **Media queries:** make sure the website components adapt to the device, so the developer does not need to maintain multiple codebases for different devices.
- **Consistency across pages:** improves the user experience and makes the website easier to navigate.
- **Accessibility:** makes sure that anyone can use the website.
- **Test early, test often:** makes sure the website functionality works well, which reduces effort and cost.
- **Title tag:** improves SEO and sets the title shown in the browser tab.
- **SEO indexing:** makes the website more discoverable and visible to users.
- **SEO meta tags:** improve the search engine rank and make it easier for search engines to index the website content.
- **SEO image names and alt attributes:** help search engines understand what the images show and make the images accessible to users.

3.3.2 Techniques Used in My Website

I used responsive design and media queries, which let me write one version of the code that works on all screen sizes. I also kept the pages consistent: all the links were implemented correctly, which makes it easy to navigate between pages.

I applied "test early, test often", which helped me fix issues early, confirm that all the functionality worked and reduce effort. I used the title tag on all pages so users know which page they are on, for example the home page title is "Takke" and the cart page title is "My Cart". I also used SEO meta tags, such as the description, to improve the search engine rank.

3.4 Website Development Frameworks

A framework is a technology that offers pre-built libraries, a way to create web applications, reusable components and built-in functionality.

3.4.1 Available Frameworks for Website Design and Development

- **Bootstrap:** a front-end framework that provides responsive, reusable components, reduces development time and keeps components looking professional across devices.
- **React.js:** a front-end library for building dynamic user interfaces. It improves component reusability and simplifies maintenance.
- **Angular:** a front-end framework for building dynamic web pages. It provides maintainability and simplifies the development of large websites.
- **ASP.NET:** a back-end framework for APIs and web applications. It has a high-performance back end and simplifies maintenance.

- **Django:** a back-end framework for building protected and secure web applications. It simplifies back-end development and provides an admin panel.
- **Laravel:** a back-end framework for building web applications with a clean syntax. It simplifies back-end development and improves productivity with libraries and built-in components.

3.4.2 Frameworks Used in My Website

I used one framework, Bootstrap, and specifically its card component. I used it to display the courses that users can add to their cart, in a clean and clear way. The card gave me a responsive design for this part without writing media queries for it, so the cards display well on all devices. It also reduced the development time and made the code easier to maintain and update.

4. Search Engine Optimisation

4.1 Search Engine Optimisation Definition and Purpose

Search Engine Optimisation (SEO) is the process of optimising a website so its pages rank higher in search engines, which makes them easier to find and more popular in user searches.

A higher rank in search results leads to more visitors. SEO also shapes the website design, navigation and content so the website is more user-friendly, attracts the target audience, and stays competitive with other websites.

4.2 Indexing Definition

Indexing is a concept in SEO. It is the process, done by search engines, of collecting, analysing and storing information from a website in a database.

Only indexed web pages appear in search engines. Indexing helps search engines understand the content of a page and know when it has been updated. Its advantages are that pages become visible in search engines, indexed pages can rank higher, and updates are reflected in the search results.

4.3 Techniques for Improving the Ranking

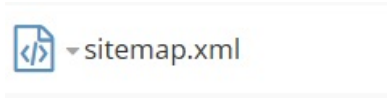
Several techniques can improve a website's ranking, and they are important before publishing a website:

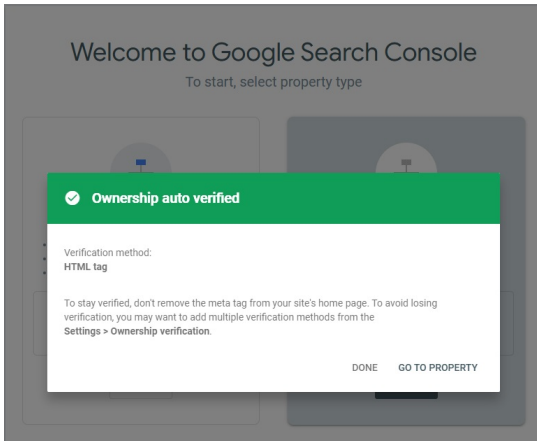
1. **Set up Google Search Console:** a tool that helps developers monitor, maintain and troubleshoot the website's presence in Google Search. The steps are creating an account, adding the website URL, submitting a sitemap and monitoring the performance.
2. **Create an XML sitemap:** a file that lists all the pages of the website, which helps search engines index them. The steps are generating the sitemap with a tool, then submitting it in Google Search Console and adding it to the code files.
3. **Include the title tag:** it is the title shown in the browser tab and in search engines.

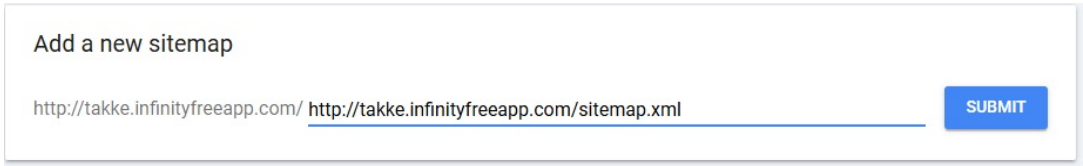
4. **Meta description tag:** a summary of what the page contains, shown in search engines below the title. It should be unique and should attract users to open the page.
5. **Use of headings:** headings improve readability. Each page should have only one H1 tag, and the other levels are used for sections and subsections.
6. **Use of the alt attribute:** it describes an image, which helps screen readers and improves the user experience.
7. **Fix broken links:** broken links lead to non-existent pages and lower the website rank, so all links should work correctly.

4.4 Evidence for the SEO Techniques Used in My Website

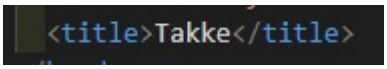
1. **Set up Google Search Console and create an XML sitemap:** I set up Google Search Console and submitted a sitemap to it.



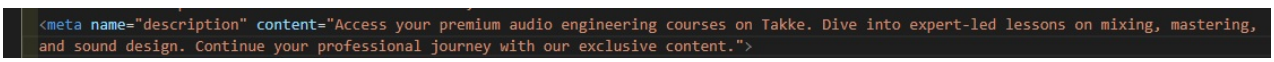




2. **Title tag:** I used the title tag on every page, with a unique one-word title for each page.



3. **Meta description tag:** I used the meta description tag on every page, with a unique short description for each page.



4. **Use of headings:** I used several headings on each page and made sure there is only one H1 per page. For example, "Takke School" is the H1 and "About The Instructor" is an H2.




```
    <div class="H1H">
      <h1>Takke School</h1>
    </div>
    <div class="Hparagraph">
      <p>Learn Audio Engineering From
    </div>
  </header>
  <main>
    <div id="about">
      <div class="Htwo">
        <h2>About The Instructor</h2>
      </div>
```

5. **Use of the alt attribute:** I used the alt attribute for every image I added to the website.

```
<div class="headerImg">
  <img src="../Assets/Takkeh.png" alt="Takke logo - Premium Audio Engineering Courses">
</div>
<div class="H1H">
  <h1>Takke School</h1>
</div>
<div class="Hparagraph">
  <p>Learn Audio Engineering From Zero to Hero</p>
</div>
</header>
<main>
  <div id="about">
    <div class="Htwo">
      <h2>About The Instructor</h2>
    </div>
    <div class="AboutDetails">
      <img class="imgAbout" src="../Assets/instructor.png" alt="Takke audio engineering course instructor">
```

6. **Fix broken links:** all links in my website work correctly, and I tested each one.

```
<?php
if (isset($_SESSION['userName'])) {
  $user = $_SESSION['userName'];
  $checkQuery = "SELECT * FROM cart WHERE userName = '$user'";
  $checkResult = mysqli_query($con, $checkQuery);
  if (mysqli_num_rows($checkResult) > 0) {
    echo "
<a href='#courses'> <i class='fa-solid fa-store'> </i> Courses</a>
<a href='#about'> <i class='fa-solid fa-address-card'></i> About</a>
<a href='mycourses.php'> <i class='fa-solid fa-chalkboard-user'></i> My Courses</a>
<a href='cart.php'><i class='fa-solid fa-cart-shopping'></i>  My Cart </a>
<a class='headerAicon' href='editUserInfo.php'> <i class='fa-solid fa-user'></i> My Profile</a>
<a href='logout.php'><i class='fa-solid fa-power-off'></i> Logout</a>";
  } else {
    echo "
<a href='#courses'> <i class='fa-solid fa-store'> </i> Courses</a>
<a href='#about'> <i class='fa-solid fa-address-card'></i> About</a>
<a href='cart.php'><i class='fa-solid fa-cart-shopping'></i>  My Cart </a>
<a class='headerAicon' href='editUserInfo.php'> <i class='fa-solid fa-user'></i> My Profile</a>
<a href='logout.php'><i class='fa-solid fa-power-off'></i> Logout</a>";
  }
} else {
  echo "
<a href='#courses'> <i class='fa-solid fa-store'> </i> Courses</a>
<a href='#about'> <i class='fa-solid fa-address-card'></i> About</a>
<a href='signin.php'> <i class='fa-solid fa-chalkboard-user'></i> My Courses</a>
<a href='signIn.php'><i class='fa-solid fa-right-to-bracket'></i> Sign In </a>
<a href='signUp.php'><i class='fa-solid fa-right-to-bracket'></i> Sign Up </a>";
}
?>
```

5. Front-End and Back-End Website Technologies

5.1 Front-End and Back-End Definition and Related Technologies

The front end is the user side of the website. It is everything the user sees and interacts with directly, such as the content, elements, styles and colour palette. Front-end technologies include HTML, CSS, JavaScript and frameworks such as Bootstrap.

The back end is the server side of the website. It is everything the user does not see, and it

handles the data: inserting, updating, reading and deleting it. Back-end technologies include PHP, databases, APIs and frameworks such as Laravel.

5.2 Presentation and Application Layers Definition

The presentation layer is responsible for the user interface and for presenting data on the user's device. It is where the user interacts with the application through the graphical user interface. Common technologies are HTML, CSS and JavaScript.

The application layer, also called the logic tier or middle tier, sits between the presentation layer and the data layer. It is the heart of the application: it processes the information collected from the presentation layer and is allowed to add, modify or delete data in the data layer. Common technologies are Python, Java and PHP.

5.3 Relationship

The front end and the back end work together to create a fully functional website, with the visible part and the hidden part. The front end manages the user interface, the interaction and what the user sees, while the back end manages the data. For example, when a user wants to create an account, they see a sign-up page, and when they press the sign-up button, a hidden process in the back end inserts the entered data into the database.

The front end relates to the presentation layer, since both are the part the user sees: the presentation layer formats the data and displays the interface. The back end relates to the application layer, since the application layer collects the information filled in on the front end and processes it, and the back end stores it.

For example, the sign-up form is written in the front end and the presentation layer formats and displays it. The user fills in the form and presses the sign-up button, the application layer collects the information, and the back end stores it in the database.

6. Online Website Creation Tools vs Custom-Built Websites

6.1 Online Website Creation Tools

Online website creation tools such as WordPress and Wix are content management systems used to create a fully operational website without coding knowledge. Components are dragged and dropped where needed, and many themes and templates are available.

- **WordPress:** based on PHP and MySQL. Anyone can use it to build a website without coding, but overloading a site with heavy themes may slow it down.
- **Wix:** a cloud-based website builder that uses drag-and-drop components. It is known for beginners and does not require coding.

These tools are easy to use for people without coding knowledge, they save time because a website can be created and published quickly, and they are affordable. They also have built-in tools such as SEO features, and some offer hosting and domain management.

They also have limitations. The developer does not have full control over the design and layout,

the designs are not unique because they depend on templates that many websites use, and there are scalability limits, security concerns and restricted functionality. The developer does not have full access to the back-end functionality or the database.

6.2 Custom-Built Websites

Unlike website creation tools, custom-built websites are developed from scratch. They require coding knowledge, but the developer controls every aspect of the website. In my opinion, they are better than online website creation tools because the developer has full control over the design and can create complex and unique layouts. The performance can also be optimised: clean code helps keep the load time fast and smooth. The developer also has full control over the functionality and can build custom features that are not possible with online website creation tools.

Tools used for custom-built websites include code editors and IDEs for writing and editing HTML, CSS, JavaScript and PHP. Visual Studio Code is one of the most popular editors because it has many features and extensions, and Atom is another. XAMPP and phpMyAdmin make the computer act as a server and a client at the same time, giving a local environment for managing databases and the back end before publishing. Browser developer tools help the developer debug and test the website. The techniques (responsive design, media queries, and test early, test often) and the technologies (HTML, CSS, JavaScript, databases and protocols) are the ones discussed earlier in this report.

When developing the website, I made decisions about the technologies, management services, tools and software to use in order to achieve its primary goal.

Technologies. I chose HTML for the structure and content, and CSS to style the content and give the user a good visual experience. I chose JavaScript to show alerts and simulate real-time validation. PHP was important for the back end because it connects the website to the database, and the database stores and manages the website data, including the users and other information. I also chose Bootstrap, and the cards displayed on the home page are Bootstrap cards, because they saved time and gave me a pre-built component.

Management services. I first used XAMPP for local development, to test the website before publishing. XAMPP includes Apache, MySQL and PHP, which gave me everything I needed to test the website locally, and it made my laptop work as a server and a client at the same time. After testing, I chose InfinityFree to host and publish the website because it provides free hosting and the features needed for publishing.

Tools and software. I chose Visual Studio Code because it offers an efficient development environment, with features and extensions that helped me write and test the code. I chose the browser tools in Firefox Developer Edition to test responsiveness, debug, and make sure the website works on all devices. The software I used was:

- Visual Studio Code
- Firefox Developer Edition
- Miro
- XAMPP

6.3 Comparison: Design Flexibility, Performance, Functionality, UX and UI

Design flexibility. Online website creation tools provide themes and templates, so unique or complex designs, and any special design requirements, cannot be fully implemented with them. With a custom-built website the developer starts from scratch and has full control over the design.

Performance. Websites built with online tools may have performance issues, because they can come overloaded and the developer may add so many components that they affect the performance. A custom-built website can be faster because the developer writes clean and efficient code and has more control over optimisation.

Functionality. Online website creation tools offer fixed functionality through plugins and libraries, and complex functionality may not be possible to implement. In a custom-built website, the developer writes the functionality from scratch and can meet specific requirements.

User experience (UX). Online website creation tools provide a good basic UX, but the customisation is poor and may not meet specific needs. A custom-built website allows the developer to customise the UX and fit it to the needs of the users.

User interface (UI). Online website creation tools do not offer a unique interface, because the same interface is used by many websites. A custom-built website can have a unique interface, with full control over the UI.

In conclusion, online website creation tools do not give the developer full control over the website, while custom-built websites do. This is a simple comparison:

	Online website creation tools	Custom-built websites
Design flexibility	Limited	High
Performance	May be slower	Faster
Functionality	Limited to plugins	Custom functionality
UX/UI	Poor	High

7. Design and Development Process

7.1 Website vs Design Document

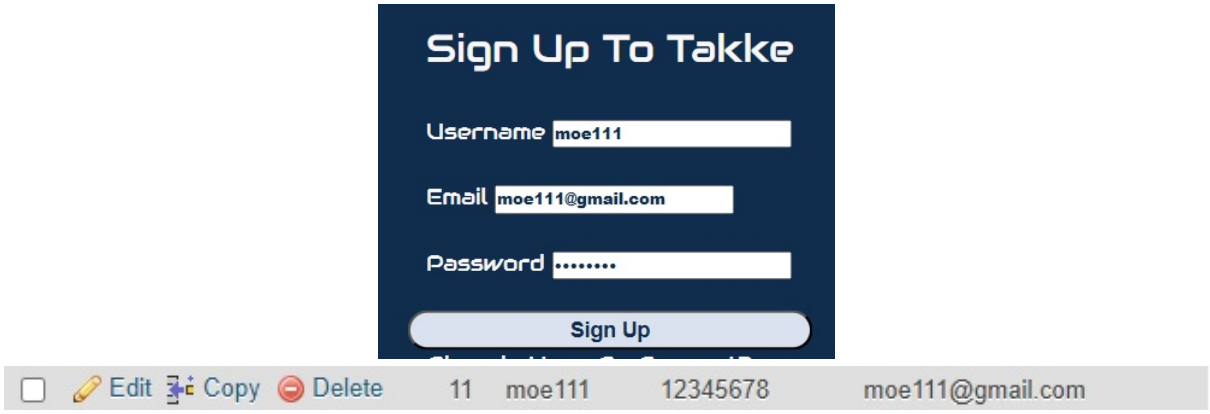
The design document describes the design of the web application: its goal, scope, requirements and what the final website will be.

My final website aligns with the goal set in the design document. Takke offers paid sound engineering courses, and the primary goal was to give students comprehensive knowledge and high skills in a field where few websites offer such courses. The website achieves this by providing courses, guidance through email, ease of access, and a clear and clean layout.

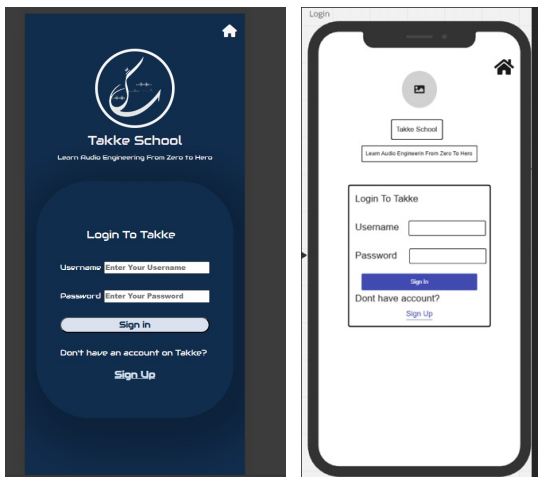
The design document also set the requirements, scope and decisions that I followed while developing Takke:

1. **User account management.** The design document explains the account management

requirements in detail. To apply them, I implemented a sign-up process that lets users create accounts and store their information in the database. The sign-in feature lets users sign in and access their profile, and the sign-out feature clears the session data. When a user signs up on the website, the data is inserted into the database, as shown below.



2. **Dynamic web pages.** The design document requires 5-6 dynamic web pages with CRUD operations. In the final website, users insert data into the database by creating an account, the courses are read from the database and displayed on the home page, users update their data from the profile page, and users delete a course from the cart page, which removes it from the cart table in the database. All of this matches the design document. I have 5 web pages in my website: the Takke home page, the cart page, the my courses page, the sign-in page and the sign-up page.
3. **Responsive design.** The design document requires the website to work well on all devices, such as phones, laptops, tablets and PCs. I used the CSS flex layout and media queries so the website matches the wireframes and works well on all devices.
4. **Wireframes and sitemap.** By applying these decisions and requirements, I created a website that matches the wireframes for mobile devices and desktops, and the website has the same pages as the sitemap.



Final sign-in page on mobile (left) and its wireframe (right)

In conclusion, the implementation of the website aligns well with the design document: the goals, scope, requirements, sitemap and wireframes all match the final website.

7.2 Evaluation and Analysis

All the design and development decisions of Takke were based on the design document and its requirements, so that the final website achieves the goals and scope.

Goal. The website goal was to create a unique learning website for aspiring sound engineers. The implementation meets this goal: it offers paid sound engineering courses, provides guidance through email, and has a clear and clean user interface.

Scope. Takke implements the scope from the design document. It delivers an educational resource through a clean layout, and it works on different screen sizes. The final website matches the wireframes and the sitemap shown earlier.

Requirements. All back-end functionality was implemented through database integration, and all five pages are connected to the database and work dynamically, as in the design document.

Challenges. I faced several challenges while designing and developing the website:

- The design and responsiveness did not match the wireframes at first, and it took me a long time to debug. The cause was that I had entered a different link when including the CSS file, so the styles did not show in my website.
- Inserting courses into the cart table was not working because my SQL query used a different table name.
- The most important challenge was translating the wireframes into a fully functional responsive website while maintaining the back-end process.

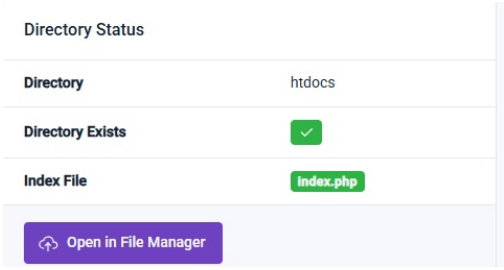
Areas for improvement. I could improve the security of the website by adding an SSL certificate, hashing the passwords and encrypting the data. I could also add tools that help students monitor their performance, such as a progress bar, assignments and more courses.

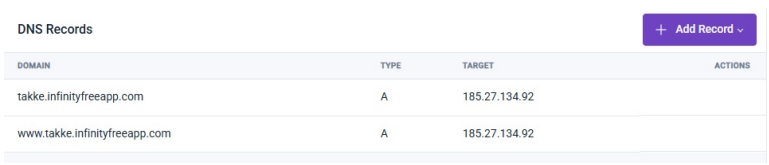
In conclusion, the design and development of Takke met the goals, scope, requirements, sitemap and wireframes of the design document. The challenges I faced were solved through debugging and testing.

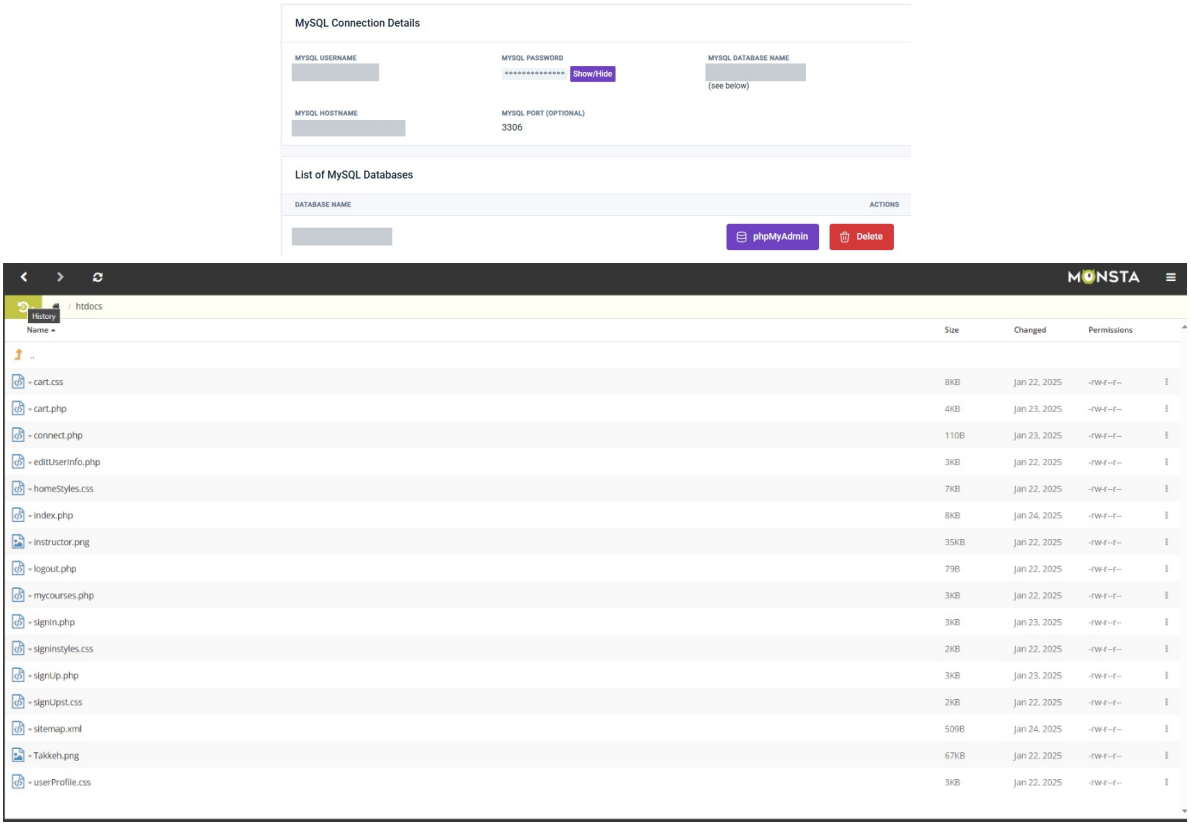
Hosting

Website: http://takke.infinityfreeapp.com/

For hosting, I used InfinityFree because it offers free hosting plans and supports PHP and MySQL databases. I configured the domain with Takke as the brand name, and I used the File Manager and FTP to upload the website files. I also configured the database in InfinityFree by creating a new database and importing my original database into it.







References

1. IBM (n.d.) DNS server. Available at: https://www.ibm.com/think/topics/dns-server
2. Wafaa, Melhem. Application layer protocols.
3. Moz (n.d.) What is SEO? Available at: https://moz.com/learn/seo/what-is-seo
4. Back4App (2023) Front-end and back-end frameworks. Available at: https://blog.back4app.com/front-end-and-back-end-frameworks/
5. Loganix (2023) What is indexing in SEO? Available at: https://loganix.com/what-is-indexing-in-seo/

Part 2: Website Design Document

Website Goal

Technological evolution has made everything digital, including learning. Takke is a website that offers paid sound engineering courses and a special learning experience for beginner and professional sound engineers. Before designing and developing the website, I needed to define its purpose, its goals, its users and their needs.

Who is Takke for? Takke is designed for aspiring and passionate sound engineers who want to upgrade their skills, and for people who want to become sound engineers. Our users share a common interest in learning and innovation in the field of sound engineering.

What do users expect to find on Takke? Users do not only browse: they are on a learning journey in sound engineering. They expect the website to provide:

- **Courses:** detailed paid courses at a good price that cover 90% of sound engineering aspects.
- **Guidance:** expert guidance by connecting with the instructor through his LinkedIn profile and by sending him emails.
- **Ease of access:** a design that works well on every device, and an easy registration and buying process.
- **A user-friendly website:** a clear and clean layout that attracts users.

Aim of the website. Takke offers a paid service, sound engineering courses. Its primary goal is to give users comprehensive knowledge and high skills in a field where there are not many websites offering courses.

The goals of Takke are:

- Deliver high-quality courses.
- Enable students to master sound engineering by gaining knowledge and improving their skills.
- Be a unique resource in a field that does not have many resources.

Scope

The scope of Takke describes the scenarios and areas that Takke will cover, and what the finished website will be used for. Takke offers paid sound engineering courses that focus on giving users a unique learning journey.

Takke will include paid courses, guidance, ease of access and a user-friendly layout. It provides educational services for all student levels, and it also provides video tutorials and resources. Takke will include an easy registration process, and the website must work well on all devices. The website is intended for sound engineers and for people who want to become sound engineers.

Once the website is developed, Takke will be a specialised website for delivering high-quality sound engineering courses. It will help users gain sound engineering knowledge and skills, provide an engaging learning environment, and be a unique resource in the sound engineering field.

Requirements

Requirement 1: User Account Management

The website must include account management features: sign up, sign in and sign out.

- **Sign up:** users can create an account, which is stored in the database.
- **Sign in:** users start a session and log in to their account.
- **Sign out:** the user terminates the session and the session data is cleared.

Requirement 2: Dynamic Web Pages

The website must have 5-6 web pages connected to the database. The website will:

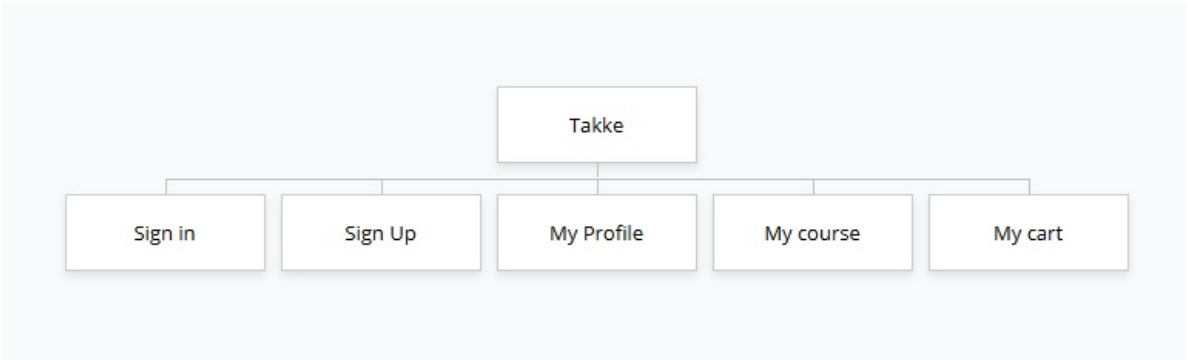
- Read the courses and the cart from the database.
- Insert courses from the website into the cart table in the database.
- Update user information from the website in the database.
- Delete a course from the cart page and from the cart table in the database.

Requirement 3: Responsive Design

The website must be suitable for all devices, such as mobile phones, laptops, tablets and PCs.

- Use flex in CSS to display the components.
- Use CSS media queries to make the design compatible with different screen sizes.

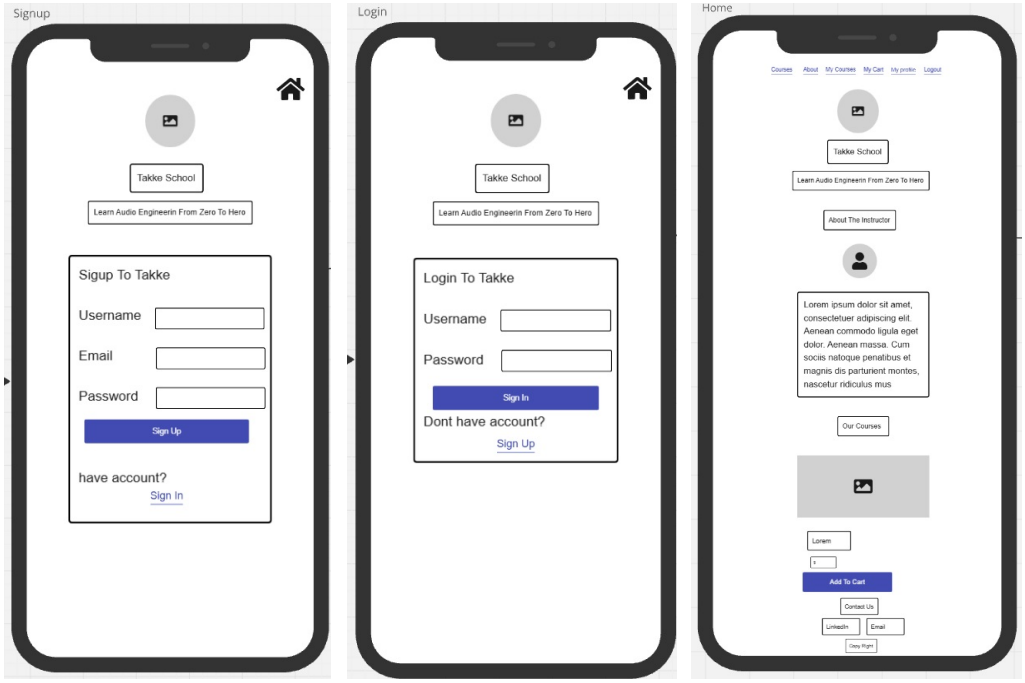
Sitemap



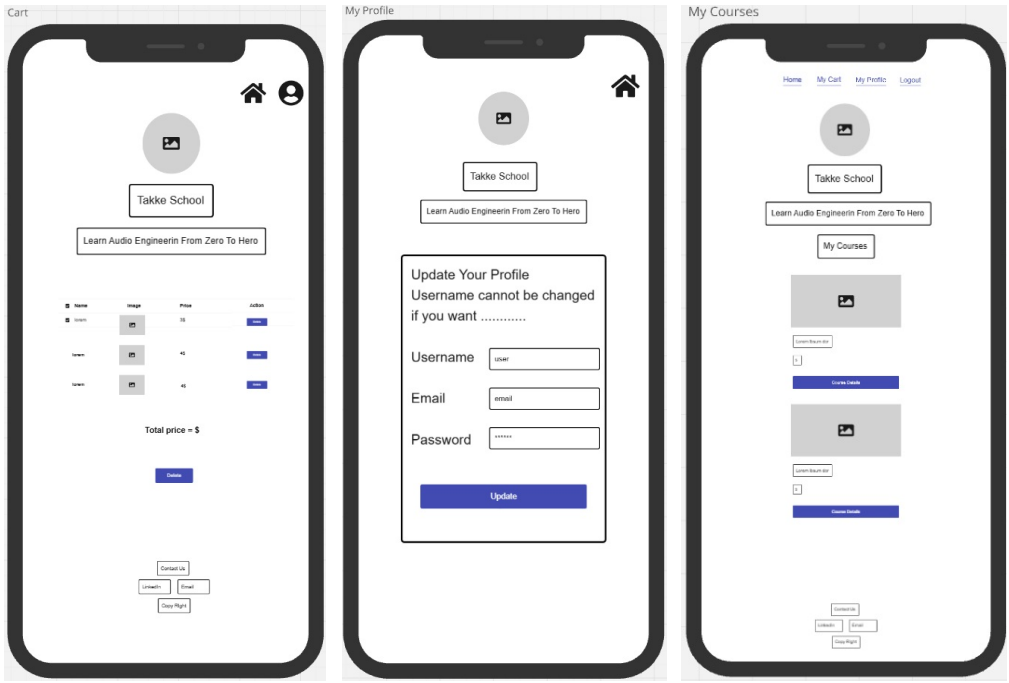
Wireframes

Wireframes for Mobile Devices

Sign up, Login and Home:

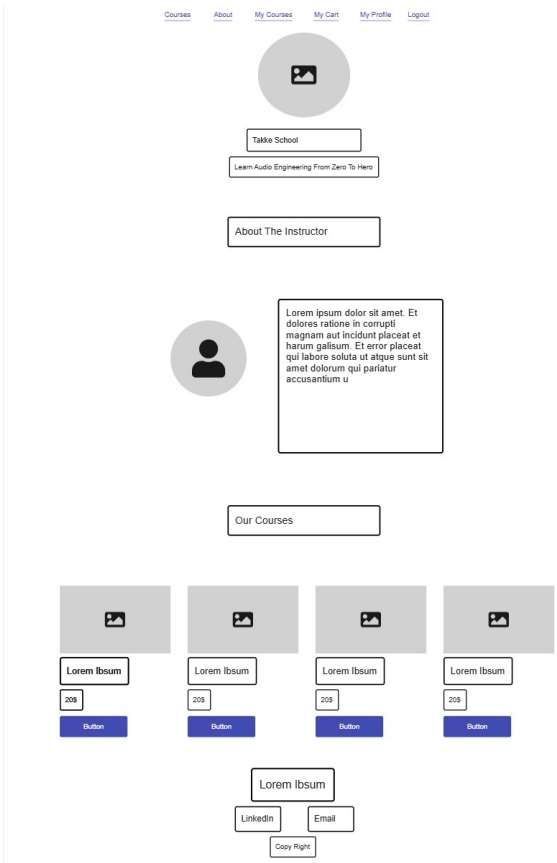


Cart, My Profile and My Courses:



Wireframes for PC Screens

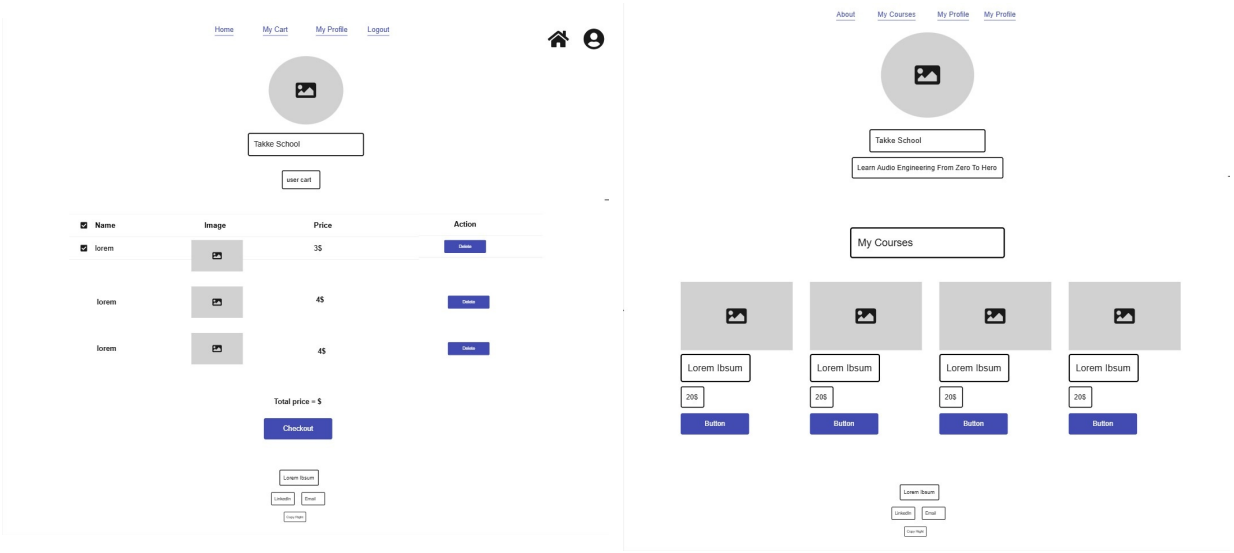
Home:



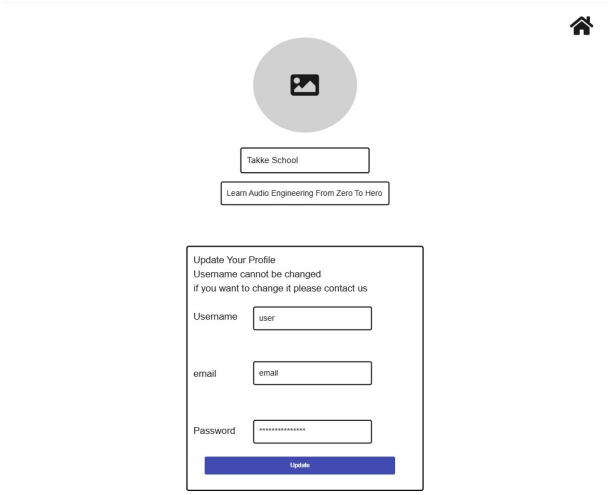
Sign up and Login:



My Cart and My Courses:



Update User Information:



Part 3: Test Plan

Scope

For Takke, I created a learning platform where users can browse and buy courses. It includes sign-in and sign-up, and it offers a learning journey for students.

In scope

- **Sign in / sign up:** users register and create an account, then log in with the information they used to sign up. The username is unique, so it identifies which user added courses to the cart and keeps each user's cart separate.
- **CRUD functionality:**
  - Create: users can sign up and add courses to the cart.
  - Read: the pages read and display the courses, my courses and the profile details.
  - Update: users can update their email and password, but not their username.
  - Delete: users can remove a course from the cart.
- **Profile update:** users can update their email and password, and the changes are saved after submitting.
- **Responsiveness:** Takke works on different screen sizes and has a flexible layout.

Out of scope

- **Payment:** there is no real checkout. When the user clicks checkout, they are redirected to the My Courses page, which displays the courses that are in the cart.
- **Security:** encryption, hashed passwords and multi-factor authentication are not included.
- **Search:** Takke does not provide a search bar for courses.
- **Custom theme:** users cannot customise the theme of the website (for example light or dark).

The scope clearly defines what is and what is not in the website, so everything is clear for the developer, the designer and the stakeholders.

Schedule

No	Date from	Date to	Hours	Test type	Description
1	25/12/2024	26/12/2024	4	Sign in / sign up validation	Test the sign-in and sign-up forms (duplicate username or email, password length, empty fields, invalid username or password) and whether an account is really created.
2	26/12/2024	28/12/2024	1.5	Content validation	Test the content shown to logged-in and logged-out users.
3	30/12/2024	30/12/2024	3	Links validation	Test that all links open their pages.
4	8/1/2025	8/1/2025	1	Icons validation	Test that all icons with a link work and appear correctly on the selected pages.
5	11/1/2025	16/1/2025	6	Functionality	Test functionality such as add to cart and checkout.
6	12/1/2025	12/1/2025	3	Responsiveness and design	Check that the designs work correctly and the pages work on all screen sizes.

Test Cases

No	Description	Type	Steps	Expected results	Status
1	Test the sign-up form with a duplicate username or email.	Sign in / sign up validation	1. Open the sign-up page. 2. Enter an existing username or email. 3. Submit the form.	A JavaScript alert appears saying "Failed, username and email should be unique!" and the form is refilled.	Pass
2	Test the sign-up form with a password shorter than 8 characters.	Sign in / sign up validation	1. Open the sign-up page. 2. Enter a password shorter than 8 characters. 3. Submit the form.	A warning appears, the sign-up process does not continue, and the user is asked to refill the password.	Pass
3	Test the sign-in form with a non-existent username or password.	Sign in / sign up validation	1. Open the sign-in page. 2. Enter a non-existent username or password. 3. Submit the form.	A JavaScript alert appears saying "Invalid username or password" and the form is refilled.	Pass
4	Test the sign-in form with a password shorter than 8 characters.	Sign in / sign up validation	1. Open the sign-in page. 2. Enter a password shorter than 8 characters. 3. Submit the form.	A warning appears, the sign-in process does not continue, and the user is asked to refill the password.	Pass

No	Description	Type	Steps	Expected results	Status
5	Test the home page content for a logged-out user (it should differ from the content for a logged-in user).	Content validation	1. Open the home page without signing in. 2. Check the result.	The logout, my profile and my cart links do not appear. Only 4 courses appear, and the sign-in and sign-up links appear.	Pass
6	Test the home page content for a logged-in user (it should differ from the content for a logged-out user).	Content validation	1. Sign in and open the home page. 2. Check the result. 3. Compare it with the logged-out result.	The logout, my courses, my cart and my profile links appear, and all courses appear. The sign-in and sign-up links do not appear.	Pass
7	Test the links for logged-in users.	Links validation	1. Sign in and open the home page. 2. Open all the links on the page.	All links work and open their page, for example the my courses link opens the my courses page and the my cart link opens the cart page.	Pass
8	Test the links for logged-out users.	Links validation	1. Open the home page. 2. Open all the links on the page.	All links work and open their page, for example the my courses link opens the sign-in page and the sign-up link opens the sign-up page.	Pass
9	Test the footer links.	Links validation	1. Open the home page. 2. Click the links in the footer.	The LinkedIn link opens the LinkedIn profile page, and the email link opens the email app with a new message to the selected address.	Pass
10	Test that the sign-up page works and creates an account.	Sign in / sign up validation	1. Open the sign-up page. 2. Fill in the form and submit. 3. Open the database and check that the data was inserted.	The data is inserted into the database.	Pass
11	Test that the sign-in page works and starts a session.	Sign in / sign up validation	1. Open the sign-in page. 2. Fill in the form and submit. 3. The user is redirected to the home page. 4. Add a course to the cart and open the my cart page.	The cart page shows the text "username Cart", for example "moe Cart" for the user moe.	Pass
12	Test the icons on the sign-in, sign-up, my cart and my profile pages.	Icons validation	1. Open each page. 2. Check that the home icon appears on all these pages and the profile icon on the cart page.	The icons are displayed and, when they act as links, the links work.	Pass

No	Description	Type	Steps	Expected results	Status
13	Test the sign-up form with a username and password but no email.	Sign in / sign up validation	1. Open the sign-up page. 2. Fill in the username and password. 3. Submit.	The message "Please fill out this field" appears on the email field.	Pass
14	Test the sign-up form with an email and password but no username.	Sign in / sign up validation	1. Open the sign-up page. 2. Fill in the email and password. 3. Submit.	The message "Please fill out this field" appears on the username field.	Pass
15	Test the sign-in form with a username but no password.	Sign in / sign up validation	1. Open the sign-in page. 2. Fill in the username field. 3. Submit.	The message "Please fill out this field" appears on the password field.	Pass
16	Test the sign-in form with a password but no username.	Sign in / sign up validation	1. Open the sign-in page. 2. Fill in the password field. 3. Submit.	The message "Please fill out this field" appears on the username field.	Pass
17	Test that the add to cart button adds the course to the cart table.	Functionality	1. Sign in. 2. Open the home page and go to the courses. 3. Press add to cart on any course.	The data is inserted into the cart table in the database for that user.	Pass
18	Test the add to cart button when the same course is added more than once.	Functionality	1. Sign in. 2. Open the home page and go to the courses. 3. Press add to cart on a course, then press it again.	A message "Course Is Already Added" appears.	Pass
19	Test that the cart page shows the courses in the cart table.	Functionality	1. Sign in. 2. Add a course to the cart. 3. Go to the cart.	The cart shows the courses that were added.	Pass
20	Test that the cart page calculates the total price correctly.	Functionality	1. Add multiple courses. 2. Go to the cart page and check the total.	The total is printed correctly.	Pass

No	Description	Type	Steps	Expected results	Status
21	Test that the logout link terminates the user session.	Functionality	1. Sign in. 2. Go to the home page and press logout.	The session is terminated and the page for logged-out users is shown.	Pass
22	Test the cart with multiple users: does each user have their own cart?	Functionality	1. Sign up and create two accounts. 2. Add multiple courses for the first and the second user. 3. Compare the carts.	Every user has their own cart.	Pass
23	Test the checkout functionality in the cart page.	Functionality	1. Add courses to the cart. 2. Press checkout.	The checkout should complete the payment and insert the cart data into an owned courses table. Because payment is not handled, checkout opens the My Courses page, which takes the courses from the cart table.	Fail
24	Check that when a user updates their profile, the database is updated.	Functionality	1. Sign in and go to the my profile page. 2. Change the email and password.	The data is updated in the database for this user.	—
25	Check that when a user deletes a course from the cart, it is deleted from the database.	Functionality	1. Add a course to the cart. 2. Go to the my cart page. 3. Press delete for a specific course.	The course is deleted from the database for this user.	Pass
26	Check that the home page reads the courses from the database correctly.	Functionality	1. Go to the home page. 2. Scroll down to the courses.	The courses are displayed on the web page and were read from the database.	Pass
27	Check that the design of the pages works correctly.	Responsiveness and design	1. Open every page. 2. Compare the design with the CSS code.	The design works correctly.	Pass
28	Check that the pages work on different screen sizes.	Responsiveness and design	1. Open every page. 2. Press inspect. 3. View the website on multiple devices.	The pages work correctly at all screen sizes.	Pass

No	Description	Type	Steps	Expected results	Status
29	Check that the design matches the wireframes.	Responsiveness and design	1. Open every page. 2. Open the wireframes. 3. Compare every page with its wireframe.	Every page matches its wireframe.	Pass
30	Check that the font is the same on all pages.	Responsiveness and design	1. Open every page. 2. Compare the font on the pages with the font used.	Every page has the same font (consistency).	Pass

Evaluation of the Quality Assurance (QA) Process

Requirements

Achieved requirements

- **User account management:**
  - Sign up: users can create an account, store their data in the database, and sign in with that data.
  - Sign in: users establish a session by signing in.
  - Sign out: users can terminate the session and sign out of their account.
  - Update user information: users can update their data in the database, such as the email and password.

Achieving this requirement was crucial because sign up is the core functionality of the website. Users need to create accounts and sign in to establish a session and to keep their progress private. Sign out is a security feature that lets the user terminate the session and log out. Updating information improves the user experience by keeping the data up to date. Without this requirement the website would not be useful.

- **Dynamic 5-6 web pages with CRUD operations:**
  - Read: users see the courses in the courses section of the home page, retrieved from the database.
  - Insert: users insert data into the database by creating an account and adding courses to the cart.
  - Update: users update their information in the database, such as the email and password.
  - Delete: users delete a course from the cart, which also deletes it from the database.

Reading the courses from the database means that a new course does not have to be added manually in the code, which would waste time. Inserting data is important for data management, since users can create accounts and add courses to their cart, and without it users could not have a private experience. Updating and deleting data lets users manage their account and their cart and change their decisions about the selected courses.

- **Responsive design:** users can open Takke on different screen sizes and the website displays well on them. I used flexbox and media queries to ensure this. With so many devices and screen sizes today, responsive design makes sure Takke works correctly on them, which improves usability and consistency.

Requirements not achieved

- **Checkout:** a real checkout process is not implemented. Instead, when the user presses checkout, the website goes to the my courses page, which reads data from the cart table.
- **Custom themes:** custom themes such as light and dark mode are not implemented, because the website has a single navy theme.
- **Some security measures:** encryption, hashing and multi-factor authentication are not implemented, because they are advanced and I will include them in the future.

The achieved requirements are the core of Takke's functionality. The requirements that were not achieved were left out because of their complexity or because they were outside the scope.

Design

The requirements of Takke are reflected in its design, and they were converted into a visual and user-friendly website:

- **User account management:** the sign-in, sign-up and update-information features are designed as clean forms with labels, inputs and input validation, so users can easily see how to create an account, log in and update their profile information. The logout is a button in the navigation bar.
- **Dynamic 5-6 web pages:** the home page shows different content to logged-in and logged-out users, which encourages visitors to create an account to see the hidden content. The courses on the home page have an add to cart button with an effect that attracts the user's attention. The delete button on the cart page is placed beside each course so unwanted courses can be removed, and it also has an effect.
- **Responsiveness:** I used the flexbox system and media queries to make sure the website looks good on all devices, which keeps it accessible and user-friendly.

The color palette used was: https://colorhunt.co/palette/f9f7f7dbe2ef3f72af112d4e

Front-End and Back-End Development

16 test cases relate to front-end development: 2, 4, 5, 6, 7, 8, 9, 12, 13, 14, 15, 16, 27, 28, 29 and 30. They cover input validation, the content for logged-in and logged-out users, links, icons, the design on different screen sizes, the wireframes and the font.

14 test cases relate to back-end development: 1, 3, 10, 11, and 17 to 26. They cover duplicate usernames and emails, creating an account, starting and ending a session, adding to the cart, calculating the total price, a separate cart for every user, checkout, updating the profile, deleting from the cart, and reading the courses on the home page.

Critical Evaluation of the Test Plan Results

The test plan had 30 test cases. 29 test cases passed and 1 failed (test case 23), which gives a success rate of 96.7% and a failure rate of 3.3%. I based the tests on the requirements to make sure every requirement was implemented. The high success rate shows that most of the functionality, content, design, validation and requirements were implemented correctly. The

failed test case is outside the scope of the website, so it is not important for now.

The test plan was successful for user account management, CRUD operations, responsiveness and design, front-end and back-end validation, database operations, dynamic web pages, and icons and links. The main weaknesses are the checkout, some security measures, the search functionality and custom themes (light or dark). There are also minor features that are in neither the test plan nor the website, such as a wishlist, user ratings and performance optimisation. All of these should be considered for the future.

Review of the Overall Success Areas

- **User account management:** users can create accounts, sign in and start a session, log out and end the session, and update their account details in the database.
- **CRUD operations:** users add courses to their cart (inserted into the cart table), view the courses, their profile and their cart (read from the database), update their profile details (updated in the database) and delete a course from the cart (deleted from the database). Without CRUD operations and user account management, the website would be useless.
- **Responsiveness and design:** Takke works well on laptops, PCs, tablets and mobile phones, the design matches the wireframes, and the colour palette was chosen effectively.
- **Dynamic web pages:** navigation between the pages is easy, and logged-in users see different content from logged-out users, which encourages visitors to create an account.
- **Validation, database operations, icons and links:** the front-end and back-end validation works, the database operations (insert, update, read and delete) are correct, and the icons and links give smooth navigation between pages.

Recommendations for Improvement

- **Checkout:** the most important improvement is a real checkout system that lets users complete their purchase. The current checkout button only redirects the user to the my courses page.
- **Security:** implement password hashing, data encryption and multi-factor authentication. I should also add a way to reset a forgotten password by email, which reduces the risk of unauthorised access.
- **Course progress:** track the user's progress in each course, which makes the learning journey more efficient and keeps users engaged.
- **Course details:** add a details page for every course showing its content, with user reviews and ratings.
- **Roadmaps:** provide a roadmap for each course, and a roadmap for learning sound engineering from beginner to professional.
- **Other features:** a search function for courses, custom themes such as light and dark mode, a wishlist to save courses for later, certificates for users who finish a course, and assignments to make the learning journey more engaging.

In conclusion, Takke has more strengths than weaknesses, with a 96.7% success rate.

Improvements such as the checkout system should be considered to bring it to 100% and keep Takke competitive in its field.